

Systems Overview: Multi-Round Sequential DRL-AMR

The Three Components

There are three pieces of software that work together. Think of them as three layers, each with a clear job.

The Solver

The solver is a 1D advection equation solver built on the nodal Discontinuous Galerkin method — the same kind of solver we would use with or without reinforcement learning. It knows how to advance the PDE forward in time on a given mesh. It also has the mechanical ability to adapt the mesh: split an element into two children, merge two siblings back into their parent, and enforce 2:1 balance constraints across the mesh tree. But the solver has no native AMR criterion — no built-in logic for deciding *which* elements should be refined or coarsened. If you ran the solver by itself, nothing would ever get adapted. That decision-making gap is exactly what the RL agent fills. In a traditional workflow, you'd plug in a threshold-based error indicator to fill the same role. The solver doesn't care *who* is making the decisions or *how* — it just executes mesh operations on command.

The Environment

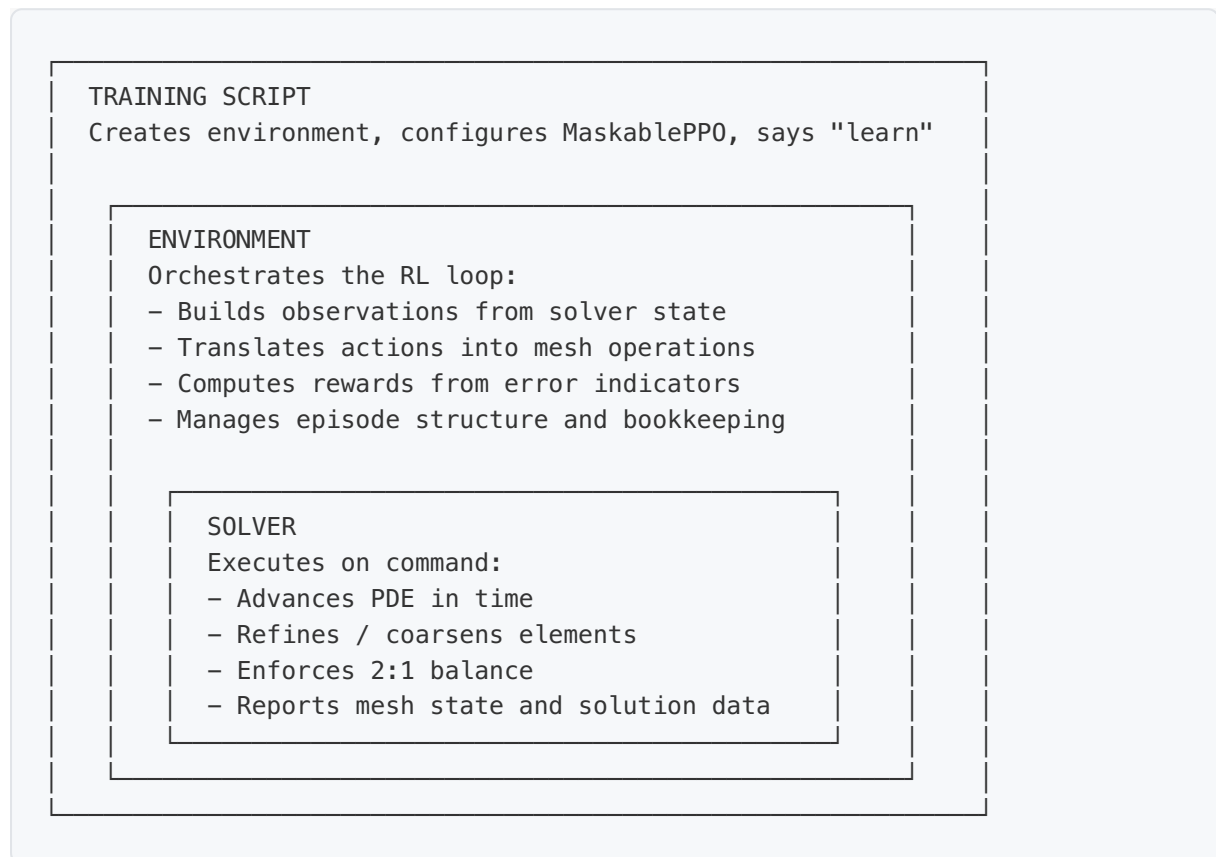
The environment is the orchestrator and translator. It sits between the RL agent and the solver, and its job is to turn the RL problem — “learn to make good mesh decisions” — into a sequence of concrete interactions the agent can handle. The environment *owns* the solver. It creates it, calls it, and reads from it. When the agent needs to see the current state of the mesh, the environment queries the solver, computes error indicators, normalizes everything, and packages it into an observation vector. When the agent picks an action, the environment translates that into a mesh operation, tells the solver to execute it, enforces 2:1 balance, and then evaluates whether the action was good or bad — that evaluation is the reward. The environment also manages all the bookkeeping: which element is the agent looking at right now, what round are we in, when is it time to advance the PDE, when does the episode end. All the structure of the training loop — the nesting of intervals, rounds, and element visits — lives here.

The Training Script

The training script is the manager. It creates the environment, configures the RL algorithm (MaskablePPO), and says “go learn.” The algorithm interacts with the environment through a

standard interface — get an observation, pick an action, receive a reward — and uses that experience to update the policy network's weights. The training script also handles logistics: saving checkpoints, logging to TensorBoard, generating diagnostic reports. It doesn't know anything about meshes or PDEs. It just knows it's training a policy to maximize cumulative reward in some environment.

Data Flow During Training



The nesting reflects ownership: the training script owns the environment, the environment owns the solver. Information flows inward (actions) and outward (observations, rewards), but the solver never communicates directly with the training script.

The Gymnasium Interface

The environment follows the Gymnasium interface standard — a standardized contract for how RL environments communicate with RL algorithms. The contract is minimal. The environment must do two things:

- **reset()** — Start a new episode and hand the agent an initial observation.
- **step(action)** — Accept an action, and return the next observation, a reward, whether the episode is over, and any extra information.

That's the entire contract. Think of it as a standardized plug and socket. Our environment is a very specific appliance — it's a DG mesh adaptation problem with all the physics and numerics that entails. But because it has the standard plug, any compatible RL algorithm can connect to it. We could swap MaskablePPO for a different algorithm without changing a single line in the environment. And conversely, someone could take our RL algorithm and plug it into a completely different environment — a video game, a robot, a traffic controller — without changing the algorithm. Everything domain-specific (the DG solver, error indicators, 2:1 balance, queue ordering, reward computation) is encapsulated inside the environment.

The Surgery Metaphor

The solver is a **surgical instrument** — it can cut precisely, suture, and cauterize, but it doesn't decide where to operate. The environment is the **operating theater** — it positions the patient, monitors vitals, presents the surgeon with the relevant imaging at each decision point, and evaluates outcomes. The agent is the **surgeon in training** — learning through thousands of procedures which situations call for which interventions. The training script is the **residency program** — it structures the training, tracks progress, and updates the curriculum.

The surgeon (agent) never touches the patient directly — every intervention goes through the instrument (solver), orchestrated by the theater (environment).

Key Points

1. **The solver is deliberately passive.** It executes mesh operations on command but makes no decisions. It is the same solver we'd use in any non-RL context — just without a traditional AMR criterion plugged in.
2. **All intelligence lives in the environment.** Observation construction, reward computation, episode structure, queue management, action masking — everything that makes this an RL problem is encapsulated here.
3. **The Gymnasium interface makes the RL algorithm swappable.** The training script and RL algorithm are completely generic. Every domain-specific detail is hidden behind the `reset()` / `step()` contract.